Abilian
CS LAB

TECHNICAL REPORT TR-02

Hop3: Consolidation, Operability, and the Road to a Final Release

Abilian SAS · June 2026 · 1.1 (SECOND INTERIM)

Contact hop3@abilian.com
Project Nix Integration for Hop3, NGI0 Commons Fund
Status Second interim technical report. It complements TR-01 (April 2026) without restating it, and covers the 0.5 and 0.6 development cycles. The final NGI deliverable release is 0.7, in preparation as this report is written; a final report with the quantitative evaluation accompanies it and remains scheduled for late 2026.

Abstract

This second interim report covers the period from TR-01 (April 2026) across the 0.5 and 0.6 development cycles. Where TR-01 established Hop3′s architecture and the Nix-based reproducible build path, the 0.5 cycle had a theme of *consolidation and operability*: turning a system that could deploy a broad application catalogue into one whose failures are legible, whose privileged operations are confined behind a narrow boundary, whose application resources are declared, and whose control surface is coherent. We describe five subsystems that matured or were introduced in that window: a unified testing architecture with a failure-diagnosis discipline (ADR 043) and a nightly Test Lab (ADR 044); a privileged-operations agent (`hop3-rootd`, ADR 041) and an exclusive-host-port registry (ADR 045) that together give Hop3 a defensible network-security and privilege-separation story; a declarative application-resource model (ADR 046) and a server configuration and secret-storage scheme (ADR 048); a redesigned command-line surface and context model (ADRs 036, 042, 047); and a central catalogue-distribution mechanism (ADR 049). The 0.6 cycle that followed both extends the platform (per-app resource limits and volumes, ADR 046 Phase 2; an operational addon-management surface; and a signed app-catalogue distribution mechanism, ADR 049) and completes the dissemination work: a published, accuracy-audited documentation corpus that now includes the full ADR record, narrative series on the testing architecture and on migration, and an automated cross-server backup-migration test. The final NGI deliverable release, 0.7, is in preparation as this report is written and carries the milestones that remain open: production upgrades, the WAF layer, an email addon, recorded screencasts, the external security review, and the quantitative benchmarks. We update the NGI deliverable mapping across the two completed cycles, note where TR-01′s description has since been superseded by the code, and restate the quantitative measurements that remain the gating item for the final report.

Keywords: Platform-as-a-Service, testing infrastructure, privilege separation, declarative configuration, secret management, command-line ergonomics, reproducible builds, digital sovereignty.

Contents

1. Scope and Relationship to TR-01

TR-01 documented Hop3′s design at the level of its standing architecture: the plugin decomposition, the deployment lifecycle, the configuration model, the Nix integration and its runtime contract, plus a preliminary qualitative evaluation across roughly one hundred end-to-end application-variant deployments. That material is not repeated here. The reader is referred to TR-01 §3–§4 for the architecture and to TR-01 §5 for the application-coverage evaluation, both of which remain current.

This report covers the work delivered across the 0.5 and 0.6 development cycles. Neither added a new headline capability comparable to the Nix integration; their character was different. With the build path proven across a wide catalogue, the dominant source of friction shifted from *can the platform deploy this application* to *can an operator understand, secure, upgrade, and trust a running deployment*. The 0.5 cycle (§2–§8) is the platform's answer to that second question; the 0.6 cycle (§9) extends the platform with resource controls, addon operations, and catalogue distribution while completing the documentation and design-record work. The sections below treat each strand in turn; §10 gives the updated deliverable mapping and §11 the outstanding work.

The project records its design decisions as Architecture Decision Records (ADRs). The corpus has grown from the 39 ADRs catalogued in TR-01 Appendix E to 49, the additions (040–049) corresponding closely to the subsystems described here. The full corpus is now published as part of the documentation site, so a reviewer can read any decision's context and consequences directly.

2. A Failure That Motivated a Cycle

A single class of failure motivated much of this cycle. An application could be healthy (its process running, its database reachable) and yet return a front-end 502 to every request, because the reverse proxy had been pointed at the wrong port or host. The application was fine; the integration was not. None of the platform's test surfaces caught this. An operator confronted with it had no signal that distinguished it from an actual application crash.

This "silent 502" is representative of a broader category: distributed failures whose cause lies at the interface between two healthy components. A single-server PaaS has many such interfaces: proxy-to-worker, worker-to-addon, builder-to-runtime, installer-to-host. The 0.5 cycle treated making them legible as a first-class goal, and three of the subsystems below (the testing architecture, the diagnosis discipline, the privilege boundary) are direct responses to it.

3. A Unified Testing Architecture

3.1 Motivation

TR-01 §5.2 described the test corpus as four layers (unit, integration, system, and end-to-end). An audit during this cycle found that this taxonomy had decayed in practice. The "system" layer (`c_system`) had accumulated almost no live tests; the real boundary in day-to-day work was between in-process integration tests and full Docker-backed end-to-end deployments. Several parallel test entry points (a bespoke `demos` harness, standalone tutorial scripts, the `hop3-test` runner, a documentation-execution path) had each been added for a good local reason, with no unifying model and with a meaningful fraction no longer working. The defaults were slow. The diagnostic coverage was inverted: the richest failure collection was wired only to the least-run path.

3.2 The Model

ADR 043 replaces this with a deliberately small model: **three runners, three layers, four tiers**, with a single rule governing the layering. A test's layer is decided by its dependencies. The three layers are `a_unit` (no Docker, counts toward coverage), `b_integration` (in-process, real in-memory database, no Docker), and `c_e2e` (Docker, real deploy, no coverage). The collapsed `c_system` layer is removed; its few live tests were redistributed to the layer their dependencies dictated. The fast unit-and-integration path runs on every commit; the Docker end-to-end layer runs on release branches and nightly. This corrects the four-layer description in TR-01: the operative architecture is now three layers. The change was a removal.

Markers are stamped from the directory layer at collection time, so selection (`-m fast`, `-m "not needs_docker"`) works from anywhere in the monorepo without per-test annotation drift, a recurrent failure mode of the previous decorative-marker scheme.

3.3 Diagnosis as a Test Concern

The structured `Diagnosis` record introduced in TR-01 §4.6 (`(component, action, reason, hint)`) is extended in this cycle into a *classifier*. On an end-to-end failure the runner now emits a headline verdict drawn from a fixed set (`proxy-502`, `build-failure`, `addon-unreachable`, `app-crash`, `timeout`) together with the decisive signals, ahead of the raw logs. The silent-502 class of §2 is one of these verdicts: a probe distinguishes "the worker is down" from "the worker is up but the proxy cannot reach it", and reports which. The per-test runtime-log collection described in TR-01 (capturing application directory, worker logs, build log, generated proxy and process configuration, and container state *before* cleanup destroys them) is now uniform across the runners rather than wired only to the remote path.

3.4 The Test Lab

ADR 044 specifies a nightly **Test Lab**: a Hop3-hosted web application that provisions ephemeral cloud servers, runs the whole suite against fresh installs, and turns each run into a queryable, trend-aware report in place of a wall of CI log output. The substrate (scheduler, runner orchestration, dashboard, report storage) is built; the persistence schema and a log-redaction pass identified as the most important pre-publication security item remain in progress. The Test Lab is the mechanism by which "the advertised application set passes on a clean blank-slate run" becomes a checkable nightly property.

This subsystem completes milestone M3.4 and is described for a general audience in a five-part documentation series accompanying the release.

4. Privilege Separation and Network Security

4.1 The Privilege Boundary

TR-01 §3.6 sketched a security model premised on a trusted operator and per-application POSIX isolation. That model left one uncomfortable gap: several operations the control plane must perform (writing reverse-proxy configuration, reloading the proxy, applying firewall rules, managing certificates) require privileges that the long-running server process should not hold. Granting the server root, or shelling out to `sudo`, are both poor answers; the first widens the blast radius of any control-plane compromise, the second is hard to audit.

ADR 041 introduces **`hop3-rootd`**, a small privileged agent that is the *only* component running with elevated privilege. The control plane, running unprivileged, asks the agent to perform a fixed, enumerated set of operations across the kernel boundary (firewall changes, proxy reload, daemon control); the agent validates each request against an allowlist and performs nothing outside it. The privileged surface is thereby reduced from "everything the server does" to "a short, reviewable list of operations" that an auditor can read end to end. The agent is packaged separately (`hop3-rootd`) precisely so its privileged code is small enough to review in isolation. (An installation subtlety surfaced and was resolved during this cycle: a hardened systemd unit with `ProtectHome=true` cannot execute an interpreter living under `/home`, so the agent is installed under a root-owned prefix.)

4.2 Exclusive Host Ports

Most Hop3 applications speak HTTP behind the reverse proxy and need no fixed port; they bind an assigned `$PORT`. A minority (a streaming server's RTMP ingest, a matrix federation port, a game server) need an exclusive, well-known host port that is theirs alone. Two such applications contending for the same unmanaged port is a platform-level failure that presents as a heisenbug: each works when deployed alone, then one fails once the other is present.

ADR 045 introduces a **fixed-port registry**: non-HTTP applications declare the host ports they require, the platform records the claim, detects conflicts up front rather than at runtime, opens the port in the firewall (through the privileged agent of §4.1), and releases it on teardown. The registry is the mechanism that lets several real applications with non-HTTP listeners coexist on one host without the operator hand-tracking which port belongs to which application. ADR 040 records the broader network-firewall and port-exposure design that the registry and the agent jointly realise.

The application-firewall (WAF) strand of milestone M3.5 (LeWAF / OWASP CRS integration with per-application policy) is at a first phase; the dynamic-policy and observability phases are carried into the 0.7 final-deliverable backlog. This is the principal incomplete item in T3, and is recorded as such.

5. Declarative Resources and Secret Storage

5.1 Declarative Application Resources

TR-01's configuration model (§3.4) covered build, run, environment, addons, and health checks. Two recurring needs were handled imperatively or not at all: an application that needs a generated secret (a Django `SECRET_KEY`, an app-specific token) and one that needs a persistent directory surviving redeploys. Operators were setting these by hand, which is error-prone and undermines the "the repository fully describes the deployment" property.

ADR 046 extends the declarative model so an application can state, in `hop3.toml`, the resources it needs: a generated secret, a dynamic environment value derived from an addon or from another field, a persistent

volume, and a resource limit. The platform realises each at deploy time. A generated secret is created once at first deploy, injected, and never regenerated on redeploy; done wrong, that last point logs every user out on every deploy. This closes the gap between “the application declares what it needs” and “the operator wires it up”, through which configuration drift enters.

5.2 Server Configuration and Secret Storage

ADR 048 settles where the server’s own configuration and secrets live, a question that had two competing answers in the code (an environment file and a TOML file) with no defined precedence, itself a source of “which one wins” surprises during redeploys. The decision: the server’s signing secret has a canonical home in a secrets-tier file (`/etc/hop3/secret-key`, root-owned, group-readable by the service user), read identically by the running service and by the privileged CLI. An environment variable remains a fallback for tests and legacy installs. Addon credentials are encrypted at rest (Fernet AEAD, key derived via PBKDF2-HMAC-SHA256 at the OWASP-recommended iteration count, with a per-install salt). The key-derivation scheme is versioned. An administrative re-encryption command migrates older installs forward onto the current parameters. A redeploy idempotence property is now explicit: a server upgrade must not rotate the signing secret or drop operator configuration.

Together with the internal security audit reported separately, this advances milestones M3.1 and M3.8. The audit’s four shipped fixes were a command-injection sweep, per-IP rate limiting on the authentication endpoints, RFC 7235 case-insensitive bearer matching and a configurable token lifetime. The external review that M3.8 also calls for and an accessibility scan both remain outstanding.

6. Command-Line Surface and Context Model

At the time of TR-01 the CLI design (ADR 036) was a draft whose command families still used the colon syntax inherited from Heroku-style tools (`app:list`, `config:set`). This cycle reworked the control surface.

The command grammar moved from colon-separated to space-separated verbs (`hop3 app list`, `hop3 env set`). The application a command targets became a uniform `--app` flag, replacing a positional argument the parser had to disambiguate per command (ADR 036). The connection-and-target model was split into two orthogonal concepts (ADR 042): a *server* is a credentialed binding to a host, while a *context* is a project’s chosen deploy target. The resolution chains for “which server am I talking to” and “which application am I acting on” were rewritten end to end. ADR 047 specifies how the resolved application and environment are transmitted with every invocation, so the server need not resolve them again. The surface gained categorised help with mandatory examples, did-you-mean suggestions, an exit-code discipline, and confirmation controls for destructive operations.

This is a correctness change: a control surface where the same concept (the target application) was expressed three different ways across commands is one where operators make mistakes, and where documentation cannot be kept consistent. The reworked surface is the precondition for the documentation effort of §8. Milestone M3.6 is complete; the CLI reference and a migration guide from the old syntax are published.

7. Catalogue Distribution

A platform whose value proposition includes a curated set of deployable applications needs a way to distribute those application specifications that does not require every operator to clone the project monorepo. ADR 049 specifies **catalogue distribution**: fetching application specs from a central source, with a producer side (publishing a catalogue) and a node side (consuming one). The node and producer

code are shipped, and during the 0.6 cycle gained catalogue signing, publisher tooling (`hop3-catalog validate` / `publish`) and an on-demand refresh against a canonical published source (§9). The install-to-deploy convenience step that would take an operator from a catalogue entry to a running application in one command remains deferred. This is a distribution channel for the packaged-application work of T4, letting an operator install a catalogued application without first understanding the project's internal layout.

8. Build Path and Application Catalogue

The Nix build path described in TR-01 §3.5 continued to mature. The source-build effort, replacing fetched prebuilt binaries with built-from-source derivations where the application is available in nixpkgs, advanced for the Go-based applications, moving several catalogue entries from Tier-3 toward Tier-1 in the reproducibility taxonomy. The template set and the runtime contract are unchanged from TR-01 Appendix B.

The application catalogue grew: at the time of writing the source tree carries 41 native-toolchain application configurations, 34 hand-crafted Nix configurations, and 31 template-generated Nix configurations, each exercised through the `hop3-test` runner, alongside the smaller fixture suites. Each addition is treated as a probe of the platform's edges rather than as a checklist item: when an application fails, we ask what the platform is missing and record the answer. Applications that cannot currently be packaged carry an explicit `DEFERRED.md` naming the blocker and the unblocker (TR-01 §5.1); these records are the project's backlog of platform gaps, deliberately public. The remaining T4 work is the formatting of these accumulated lessons into standalone per-application experience reports, and a first wave of sustained production deployments.

9. The 0.6 Cycle: Resource Controls, Addon Operations, and Dissemination

The 0.6 cycle extends the platform with new capability and completes the dissemination work in parallel. It is the last cycle before the final NGI deliverable release, 0.7, which is in preparation as this report is written and which carries the milestones that remain open.

Resource limits and volumes (ADR 046, Phase 2). The declarative-resource model of §5 gained an enforcement layer. An application declares memory and CPU caps under `[limits]`, resolved against a server-wide default and ceiling; the caps are applied to native processes through cgroup v2 (mediated by the `rootd` boundary of §4) and to containerised applications through the Docker deployer. Applications can also mount persistent bind volumes and tmpfs, provisioned by `rootd` behind a default-deny allow-list and reconciled at startup. The control surface reports the active caps and any out-of-memory terminations in `app status`, so resource pressure is legible to the operator.

Addon-management surface (T3, M3.1). The backing-service addons of §5 acquired an operational command set, uniform across PostgreSQL, MySQL, Redis, and S3: ad-hoc queries under least privilege, read-only diagnostics (process lists, locks, settings), cloning, and dump streaming through stdin/stdout for export and import, with confirmation on the destructive verbs. A second group governs access and topology: an existence predicate, per-addon variable namespacing and promotion, an endpoint accessor, server-side exposure through a `rootd`-mediated forwarder, and a developer-facing tunnel. The addon model thus covers the day-two operations an operator needs, beyond provisioning and attachment.

App-catalogue distribution (ADR 049, ADR 031). Hop3 verifies and loads an installable-application catalogue from a signed central source, refreshed on demand and at setup time, with publisher tooling to validate and sign a catalogue tarball. The dashboard renders the catalogue with untrusted content sanitised. This is the distribution counterpart to the packaged-application work of §8: the platform can offer a curated, signed set of applications rather than only build whatever source it is pointed at.

Dissemination and documentation (T5). The documentation moved from a source-tree artefact to a published, audited corpus. The full design record (now 49 ADRs) is generated into the documentation site, making every decision's context and consequences available without reading the source. The reference and developer documentation were put through an accuracy pass against the code, correcting a class of drift in which the docs described an earlier command surface and an earlier test taxonomy. Two narrative series were written: a five-part account of the testing architecture, and a "migrating from X to Hop3" series situating Hop3 against the platforms operators leave behind. This report and its predecessor are the technical-report strand of the same effort.

Resilience (T3). The cross-server backup-migration path (backing an application up on one host and restoring it on another) gained an automated end-to-end test (`packages/hop3-server/tests/c_e2e/test_backup_migration.py`), closing the gap recorded under M3.3 in TR-01′s mapping. The diagnosis discipline and the Test Lab of §3 reached the state described there during this cycle.

Build path and applications (T1/T2/T4). The source-build effort of §8 continued and the multi-component application model (ADR 038) advanced in design, while the packaged-application catalogue and its deferred-app records were maintained against the moving platform.

Several milestones remain open at the time of writing and are carried into the 0.7 final deliverable release: a production `hop3 upgrade` command (M3.2); WAF integration layered on the network firewall already shipped (M3.5); an email/SMTP addon (M3.1); recorded screencasts (M5.6); the external security review (M3.8, gated on a third-party auditor); and the quantitative benchmarks that gate the final report (§11).

10. Updated NGI Deliverable Mapping

This table updates TR-01 Appendix D to the state across the 0.5 and 0.6 cycles. "Shipped" means in the released code with documentation; "partial" means a defined subset is shipped with the remainder scheduled; "design" means a specification exists without production implementation.

Task / Milestone	State (June 2026)	Primary evidence
T1: Nix build plugins (M1.1, M1.2)	Shipped	ADR 006, ADR 008; `plugins/build/nix`; `apps/real-apps-nix`, `apps/real-apps-nix-gen`
T2: Nix runtime (M2.1, M2.2)	M2.1 shipped; M2.2 beta (≈90%); M2.3 in progress	ADR 009, ADR 035; runtime contract; `apps/bad/*/DEFERRED.md`
T3: Backing services (M3.1)	Shipped (PostgreSQL, MySQL, Redis, S3/MinIO) with a full operational command set; email deferred	ADR 046; `plugins/{postgresql,mysql,redis,s3}`; §9
T3: Resource limits & volumes (ADR 046 P2)	Shipped (cgroup-v2 caps for native + Docker; bind/tmpfs volumes)	ADR 046; `hop3-rootd`; §9
T3: Upgrades (M3.2)	Partial (schema migrations; production `hop3 upgrade` still open)	`orm/alembic`

Task / Milestone	State (June 2026)	Primary evidence
T3: Backups (M3.3)	Shipped; cross-server migration test now automated	ADR 024; `guides/backup-restore`; `c_e2e/test_backup_migration.py`
T3: Testing framework (M3.4)	Shipped	ADR 043, ADR 044; `packages/hop3-testing`; §3
T3: Firewalls / WAF (M3.5)	Network firewall shipped; WAF integration still open	ADR 040, ADR 041, ADR 045; `packages/hop3-rootd`; §4
T3: CLI (M3.6)	Shipped	ADR 036, ADR 042, ADR 047; §6
T3: Web UI (M3.7)	Basic; polish and Git-URL deploy still open	dashboard controllers and templates
T3: Security-audit outcomes (M3.8)	Code fixes shipped; external review and accessibility scan pending	§5.2; security-audit report
T4: Packaged applications (M4.1–4)	100+ configured and tested; standalone reports and production traffic in progress	`apps/real-apps-*`; §8
T4: App-catalogue distribution (ADR 049)	Shipped (signed central source, `catalog refresh`, publish/validate tooling)	ADR 049, ADR 031; §7, §9
T5: Website / blog (M5.1)	Shipped	documentation site; release and architecture posts
T5: Documentation (M5.2)	Shipped; accuracy-audited against the code, full ADR record published	guides, reference, developer docs, 49 published ADRs, tutorials
T5: Technical report (M5.3)	Two interim reports (this is the second); benchmarks pending	TR-01, TR-02
T5: Conference (M5.4)	OW2Con 2025, OSXP 2025 delivered; OW2Con 2026 scheduled	conference posts
T5: Screencasts (M5.6)	Not started; demo harness in place as the basis	`demos/`

11. Outstanding Work and Threats to Validity

The gating item for the **final** report is unchanged from TR-01 §5.4 and remains the project's principal evaluation debt: a quantitative comparison against K3s, Docker Compose, and bare uWSGI (control-plane footprint, deployment latency by build strategy, Nix closure versus Docker image size, cold-start latency, and bit-for-bit reproducibility across independent rebuilds of the Tier-1 corpus). No claim in this report is quantitative. The consolidation work described here does not substitute for that measurement.

First, the testing architecture's value is itself a claim, to be measured by the failures it now catches that the previous surface did not. The silent-502 verdict is verified by construction; the broader claim that the three-layer model improves contributor throughput is anecdotal at this stage. Second, the privilege boundary of §4 reduces the privileged surface but has not been subjected to the external security review that milestone M3.8 calls for; an enumerated allowlist is only as good as its enumeration, which an independent audit is the right instrument to check. Third, the application catalogue's breadth (§8)

remains a coverage claim, separate from operational reliability. The platform deploys a wide set; sustained production operation under real load and real upgrade paths is the validation still outstanding, and the substance of the remaining T4 work.

12. Conclusion

The 0.5 cycle turned a platform that could build and deploy a broad application catalogue into one with a coherent operational story: failures are classified before they are dumped, privileged operations are confined to a small reviewable agent, exclusive host resources are arbitrated, application needs are declared, secrets have a defined home, and the control surface expresses each concept exactly once. None of these is a headline feature in the sense the Nix integration was; together they are the difference between a system that demonstrates and a system that operates. The 0.6 cycle adds a resource-control layer (limits and volumes), an operational addon-management surface and a signed catalogue-distribution mechanism, while in parallel putting the documentation and design record into a published, audited form that a reviewer can read end to end. The final NGI deliverable release, 0.7, gathers the remaining milestones: production upgrades, the WAF layer, the email addon, screencasts, the external review, and the quantitative evaluation. That evaluation is now the clearest remaining item. The subsystems described here are the foundation on which its benchmark harness will run.

References

The architecture, related work, and application-coverage evaluation are in TR-01 (April 2026), which this report complements. The decisions described here are recorded in the project's ADR corpus, published with the documentation. The most directly relevant are ADR 036 and 042 (command surface and context model), ADR 041 and 045 (privileged agent and fixed-port registry), ADR 043 and 044 (testing architecture and Test Lab), ADR 046 and 048 (declarative resources and secret storage), and ADR 049 (catalogue distribution). Source code and documentation: https://github.com/abilian/hop3 and https://hop3.cloud.

This report was prepared as part of the NGI0 Commons Fund grant for the "Nix Integration for Hop3" project (#2024-04-365). It is the second interim deliverable; the final report, with the quantitative evaluation, is scheduled for late 2026.